

## JavaScript Modules – Import/Export & Circular Dependencies

### Basic Rule

You can use import and export freely between files, but avoid circular dependencies.

### Correct Structure

```
utils.js → test.js → index.js

utils.js:
export function sum(a, b) {
  return a + b;
}

test.js:
import { sum } from "./utils.js";

index.js:
import "./test.js";
```

### Wrong (Circular Dependency)

```
utils.js ↔ test.js

utils.js:
import { test } from "./test.js";

test.js:
import { sum } from "./utils.js";
```

### Diagram – Good Flow

```
graph TD
  index.js --> test.js
  test.js --> utils.js
```

### Diagram – Circular (Bad)

```
graph LR
  utils.js <--> test.js
  subgraph loop
    direction TB
    test.js --> utils.js
    utils.js --> test.js
  end
```

### Key Rules

- One-directional imports
- Avoid circular dependencies
- Use shared files for common logic

### Interview Tip

Circular dependencies can lead to partially initialized modules and undefined values.